I/O Models

Blocking vs Non-Blocking

Understanding the Event Loop & Node.js — with a Restaurant Analogy

I/O | Event Loop | Node.js

FOUNDATIONS

What is I/O?

I/O = Input / Output

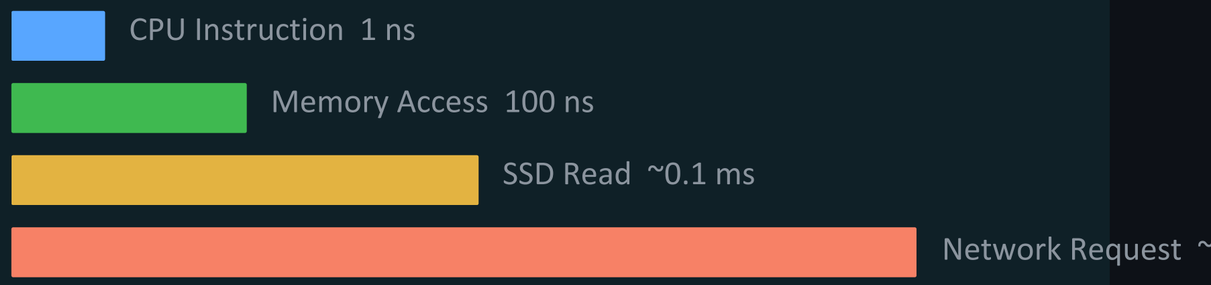
Any operation where your program talks to something **outside the CPU**:

- Reading / writing a file
- Network requests (HTTP, DB)
- User keyboard input
- Hardware sensors

Why does it matter?

I/O is SLOW compared to CPU operations. A disk read can be millions of times slower than a CPU instruction.

How your program handles that wait time defines its entire performance model.



MENTAL MODEL

The Restaurant Analogy

Think of your program as a restaurant. The kitchen = I/O (slow, external). Customers = tasks. The waiter = your thread.
q

Kitchen

Handles the actual cooking
(like disk/network doing the real work)

Slow — takes time regardless

Customer

Places an order
(like your code making an I/O call)

Expects a result eventually

Waiter

Coordinates between customer & kitchen
(your program's thread)

Can only do one thing at a time

I/O MODEL #1

Blocking I/O

The Blocking Waiter

The waiter takes an order to the kitchen…

…and stands at the kitchen window, idle, waiting for it to be ready.

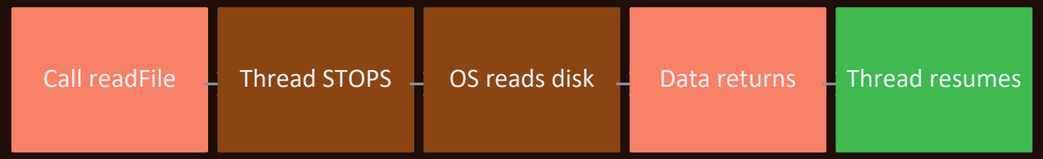
No other customer gets served during that wait.
All tables must wait in queue.

Result: Restaurant is slow. One slow order holds everyone.

In Code

```
const data = fs.readFileSync('file.txt');
// ↑ thread STOPS HERE until done
console.log(data);
```

The thread is stuck — doing nothing — while waiting for the OS to return data.



⚠ ***One slow I/O call freezes the entire program***

I/O MODEL #2

Non-Blocking I/O

The Non-Blocking Waiter

The waiter takes an order to the kitchen…

…and immediately moves to the next table, taking more orders.

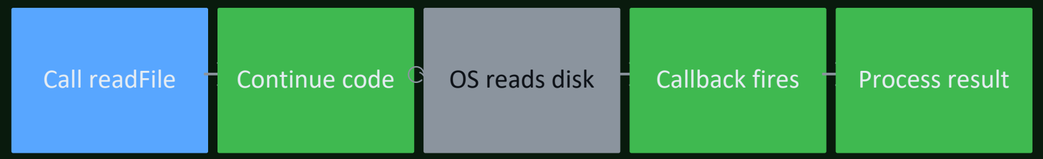
When the kitchen calls "Order ready!", the waiter goes back and serves it.

Result: All customers served efficiently. No idle time.

In Code

```
fs.readFile('file.txt', (err, data) => {
  // called LATER when ready
  console.log(data);
});
// ↓ code continues immediately!
```

The callback is the "kitchen bell" — it fires when the I/O is done. Thread is free meanwhile.



✓ ***I/O runs in background; thread stays productive***

COMPARISON

Blocking vs Non-Blocking — Side by Side

	Blocking	Non-Blocking
Thread while waiting	Idle / frozen	Free to do other work
Other tasks served?	No — must wait in queue	Yes — immediately
Code complexity	Simple, linear	Callbacks / Promises
Restaurant analogy	Waiter stands idle at kitchen	Waiter serves other tables
Suitable for	Simple scripts	Servers, high concurrency

⚠ *NOTE: Blocking/Non-Blocking ≠ Synchronous/Asynchronous. Sync/Async is about how YOUR code handles the result. These are different layers!*

COMMON CONFUSION

Sync vs Async — The Important Distinction

> *"Blocking/Non-Blocking = how the OS handles the wait. Sync/Async = how your program handles the result."*

Synchronous

Your code waits for the result
before moving on.

Does NOT mean Blocking —
you can have non-blocking sync code.

```
e.g. await fetch(url) — async code written
synchronously
```

Asynchronous

Your code fires and continues.
Result arrives via callback,
Promise, or event later.

Does NOT mean Non-Blocking.

```
e.g. fetch(url).then(cb) — async code written
asynchronously
```

Key insight: Non-blocking I/O is an OS-level mechanism. Async is your application-level strategy to use it wisely.

THE PATTERN

Event-Driven I/O

Restaurant Analogy: Instead of checking on your order every 30 seconds, you get a buzzer. When it vibrates — your order is ready. You react to the event.

Events

Signals that something happened.
e.g. 'file read done', 'request received', 'button clicked'

Emitters

Objects that produce events.
They don't know who's listening — just broadcast.

Listeners / Callbacks

Functions that subscribe to events.
They fire only when the event occurs.

Event-Driven Flow: I/O starts → Event registered → I/O completes → Event fires → Callback runs

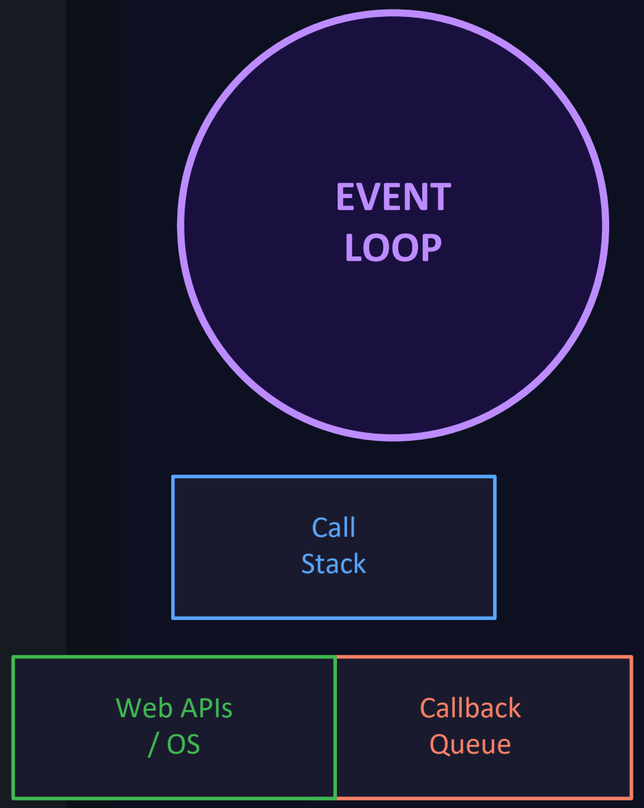
CORE MECHANISM

The Event Loop

The Waiter's Loop

The waiter does this forever:

1. Check if any kitchen bell rang
2. If yes → serve that dish
3. Take new orders from tables
4. Pass orders to kitchen
5. Repeat forever



How it works

Call Stack

Executes your code, one thing at a time.

Web APIs

Browser/Node handles I/O operations.

Callback Queue

Completed I/O callbacks wait here.

Event Loop

Moves callbacks to stack when stack is empty.

PUTTING IT TOGETHER

The Restaurant → Node.js Map

Restaurant Concept	Node.js Equivalent
One waiter	Single thread (JS engine)
Waiter takes order to kitchen	Code makes I/O call (fs, http...)
Waiter doesn't wait — serves others	Non-blocking I/O — thread stays free
Kitchen bell rings	OS signals I/O is complete
Waiter picks up & serves dish	Event loop runs callback
Multiple tables served by 1 waiter	Thousands of requests, 1 thread
Kitchen (slow cooking)	Disk / Network / DB (slow I/O)

Key Takeaways

01

Blocking I/O

Thread waits — like a waiter standing idle at the kitchen. Simple but inefficient.

02

Non-Blocking I/O

Thread continues — waiter serves others while kitchen works. I/O in background with callbacks.

03

Blocking ≠ Synchronous

These are different layers! Blocking/Non-blocking is OS-level. Sync/Async is application-level.

04

Event-Driven I/O

React to events (the "buzzer") instead of polling. Callbacks fire when I/O completes.

05

Event Loop in Node.js

Single thread, non-blocking — handles thousands of connections without extra threads.